Array Rasterizer Accelerator for a VeeR EL2 RISC-V SoC

Abstract

Problem Statement

Rasterization — converting geometric primitives such as lines and triangles into the pixel data written to a framebuffer — is a repetitive, memory-bound workload. On a conventional RISC-V SoC, the CPU must fetch each vertex, walk the edges, compute the pixels for the primitive, and write each result back through the memory hierarchy. The per-pixel computation is small, but the pixel count and resulting memory traffic are large, so the CPU spends a disproportionate number of cycles moving data rather than doing useful work.

This project builds a small hardware accelerator, the Array Rasterizer (AR), that takes over this pixel-generation work from the CPU. The goal is not to prove that hardware rasterization is always better than software — it is to measure where the tradeoff actually falls, across different primitive sizes and scene complexities, on a real VeeR EL2 SoC.

Proposed SoC Architecture

The SoC is built around the provided VeeR EL2 RV32IMC RISC-V processor and its required peripherals. The AR engine is added as an independent pair of AXI4 masters — one for reading primitive/vertex data and one for writing pixel results to the framebuffer — alongside an AXI4-Lite control interface used by the CPU.

Instead of looping over pixels itself, the CPU issues a single, compact draw command describing the primitive: endpoints or triangle vertices, colour, and destination address. The AR engine then walks the edges, generates the pixels, and writes them to the framebuffer on its own, raising an interrupt when the job is done.

A VGA controller reads the completed framebuffer and drives a standard VGA output, so the rasterizer's output can actually be seen on a display. This turns the project from a set of counters and waveforms into something demonstrable on real hardware.

The design is kept deliberately small and synthesizable — only line and triangle fill, no shading, texturing, blending, or anti-aliasing. Primitive size is treated as an experimental variable: for very small or degenerate primitives, command setup and interrupt overhead may make software competitive, while larger fills should give the accelerator enough work to justify its cost. Measuring this crossover, rather than assuming hardware always wins, is the point of the project.

Selected IPs

IP Block	Functionality
Array Rasterizer (AR)	Performs line and triangle rasterization (edge-walk / fill) on primitives read from memory, writing pixel results directly to the framebuffer.
VGA Controller	Reads the framebuffer over AXI4 and drives VGA timing/output, providing a visible demonstration of the rasterizer's output on a display.
Hardware Performance Monitor (HPM)	Records CPU cycles, memory bus transactions, pixels produced, AR active time, and setup/interrupt overhead, so software and AR-accelerated runs can be compared directly.

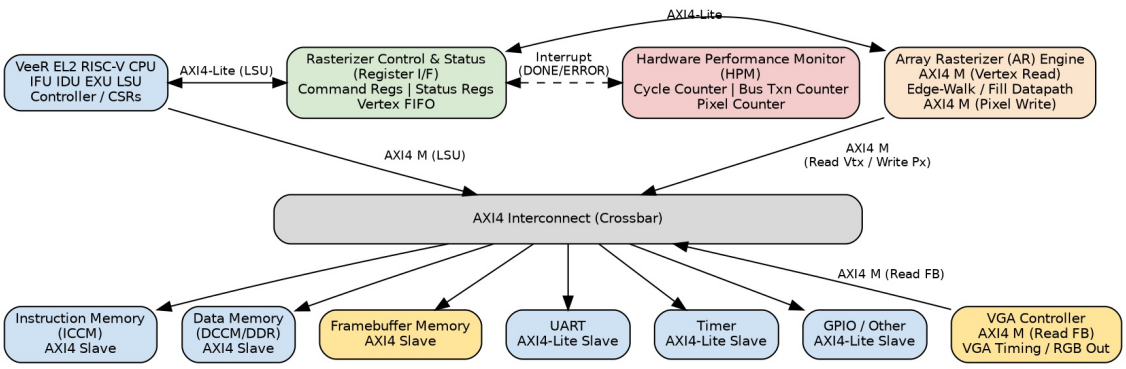
Software and Evaluation

A small software driver exposes two operations to applications running on the RISC-V processor: draw line and draw triangle. After issuing a draw command, the CPU does not need to poll the accelerator — the AR engine runs independently and raises an interrupt on completion, freeing the CPU to do other work while rasterization is in progress.

The evaluation compares conventional software rasterization against AR-accelerated rasterization across different primitive sizes and scene complexities. Measured quantities: execution time, CPU cycles, memory transactions, AR active time, interrupt/setup overhead, and FPGA resource cost (LUTs, FFs, BRAM). The aim is to identify where the AR engine helps, where software is competitive or better, and whether the measured benefit justifies the added hardware.

Architecture Document

Block Diagram



Array Rasterizer SoC Block Diagram

Array Rasterizer (AR) SoC — Block Diagram

The VeeR EL2 core is left unchanged. The AR engine is added as two additional AXI4 masters on the SoC interconnect: one dedicated to reading primitive/vertex data and one dedicated to writing pixel results to the framebuffer. It has a control interface used by the CPU and a separate memory path used to read and write array data. The CPU sends the draw command once; the AR engine handles the element-by-element (pixel-by-pixel) processing on its own. The VGA controller is a separate AXI4 master that reads the framebuffer independently and drives the display, so it does not interfere with the AR's write path.

Driver and Execution Flow

The application chooses whether to use software rasterization or the AR engine. For the AR path:

1. **Initialization:** The driver writes the primitive type, vertex/endpoint addresses, colour, destination (framebuffer) address, and start command.
2. **Autonomous Execution:** The AR engine reads the primitive data and rasterizes it independently of the CPU.
3. **Completion:** On completion, the AR engine raises an interrupt for the driver to read the result status.

Interface Specifications

The AR engine has three primary interfaces:

- **Control/Status Interface:** AXI4-Lite slave used by the CPU to configure and start operations and to read status and results.
- **Vertex/Primitive Read Interface:** AXI4 master used by the AR engine to read primitive data from memory without CPU involvement.
- **Pixel Write Interface:** AXI4 master used by the AR engine to write rasterized pixel data to the framebuffer without CPU involvement.

The control interface exposes registers for: primitive base address, vertex/endpoint count, primitive type (line/triangle), colour, framebuffer destination address and stride, and start/status.

The VGA controller has a single AXI4 master interface for reading the framebuffer, plus the standard VGA timing outputs (HSYNC, VSYNC, RGB, blanking), generated from an internal pixel-clock counter running off the framebuffer's fixed resolution.

Bus Protocol Details

- **AR Read Path:** AXI4 master with burst reads from data memory for vertex/primitive data. Once started, the AR engine generates its own read addresses and processes the returned data.
- **AR Write Path:** AXI4 master with burst writes to the framebuffer for rasterized pixel data, generated autonomously as the fill progresses.
- **AR Control Path:** AXI4-Lite slave for small, infrequent control and status accesses from the CPU.
- **VGA Read Path:** AXI4 master with burst reads from the framebuffer, paced by the VGA pixel clock rather than by the CPU or the AR engine.
- **HPM:** AXI4-Lite slave for configuration and counter/status access.
- **Completion:** The AR engine raises an interrupt when the operation completes, so the CPU can continue useful work while rasterization is running instead of polling it.

Note on Scope

A watchdog/timeout block was considered but is not included by default — the primitive set (lines, triangles) and bounded framebuffer size make an unbounded-fill hazard unlikely, and adding one would be extra hardware without a clear justification. If testing on real hardware surfaces a stall case worth guarding against, a minimal watchdog can be added later rather than built in from the start.